

CSI v10 System (AES) Using the Application Event

Study notes from the Infor official training workbook (302 pages)

Why this guide matters

The Application Event System (AES) is the automation engine inside CloudSuite Industrial. Without it, automating approvals, sending notifications, or gating a record change pending a manager's response requires writing procedural code. With it, you define metadata rules — no compilation needed — and the system handles the rest, upgrade-safely.

Use cases: auto-notify on new records, approval workflows with suspend/commit, multi-level management approvals with voting, adding information to a record via a prompted response, triggering events on a schedule or when a threshold is crossed.

Prerequisites: UI Navigation (4-unit course). Basic SQL knowledge is recommended but not required for simple notification workflows.

1. The Three Basic Components

Every AES scenario is built from the same three components, chained together:

Component	What it is
Event	A uniquely named occurrence that can be triggered by user actions, database conditions, time passing, or another event's handler. The event itself is just a name — it does nothing until a handler is attached.
Event Handler	Associated with one event. Defines: which conditions must be true, whether to run synchronously or asynchronously, whether to suspend the record pending a response, and which actions to take. One event can have multiple handlers; each handler belongs to only one event.
Event Action	The individual unit of work inside a handler. Types include: Notify, Prompt, Fail, Finish, Branch, Set Values, Load IDO Row, Update Collection, Generate Event, Call IDO Method, Call Database Method, and more. A handler must have at least one action.

Mental model: Event fires → Handler evaluates → Actions execute in sequence. If any synchronous action fails (and Ignore Failure is off), the chain stops and the handler exits with Failure status.

2. Event Types

Type	Who creates them / what triggers them
Framework Events (Access As = 'core')	Created by Infor; fire automatically when the system does something (insert a row, update a row, delete a row, load a collection, invoke a method, login/logout). You cannot create framework events, but you can attach your own handlers to them. Examples: <code>IdoOnItemInsert</code> , <code>IdoOnItemUpdate</code> , <code>IdoPostItemInsert</code> , <code>IdoPostItemDelete</code> , <code>IdoOnLoadCollection</code> , <code>SessionOnLogin</code> .

Type	Who creates them / what triggers them
Application-Specific Events	Created by Infor or business partners for specific business processes. Tagged with the owner's Access As identifier.
Customer-Defined Events	Created by your organization (Access As = blank). Can be triggered via: Event Trigger form (condition/time-based), Generate Event action type (from another handler), form event handler with 'Generate Application Event' response type, WinStudio API or .NET code.

Key insight: Framework events fire constantly in the background — every time any user saves a record, `IdoOnItemInsert` or `IdoOnItemUpdate` fires. Nothing happens unless you add a handler. The event fires, and if no handler matches, it is silently discarded.

3. The Access As Identifier

Every event, handler, and global constant is tagged with an **Access As** identifier that establishes ownership. You can only modify or delete objects whose Access As matches your current setting.

Access As value	Meaning
core	Owned by Infor's framework team. Read-only for everyone else.
Any specific name	Owned by an Infor application team or authorized business partner. You can attach your handlers to their events, but cannot edit their objects.
Blank (empty)	Owned by your end-customer organization. This is the Access As value for all the work you do in this course.

To check your current Access As: Master Explorer > Modules > System > Access As form (display-only).

4. Synchronicity — Synchronous vs. Asynchronous

Every event and every event handler is either synchronous or asynchronous. This is the most important behavioral distinction in the AES.

Mode	How it runs
Synchronous	Runs in the same thread as the action that triggered it. Blocks the user/process until all synchronous handlers complete. A failure stops the chain and can roll back the originating transaction. Framework events are always synchronous.
Asynchronous	Queued to the Infor Framework Event Service (utility server). The originating thread continues immediately after successful queuing. Runs independently — failure does not affect the originating thread. Use for notifications that should not block the user.

How to make an event sync or async

How the event is generated	Sync or Async?
Framework (core) event	Always synchronous
Form event handler: Generate Application Event, Synchronous checkbox ON	Synchronous
Form event handler: Generate Application Event, Synchronous checkbox OFF	Asynchronous
Stored procedure: EXEC FireEventSp	Synchronous
Stored procedure: EXEC PostEventSp	Asynchronous
.NET: FireApplicationEvent(Synchronous=True)	Synchronous
.NET: FireApplicationEvent(Synchronous=False)	Asynchronous
Event action: Generate Event, SYNCHRONOUS(TRUE)	Synchronous
Event action: Generate Event, SYNCHRONOUS(FALSE)	Asynchronous

Handler-level setting: Individual handlers also have a Synchronous checkbox. Clearing it queues the handler asynchronously even when the event itself is synchronous — useful for notifications attached to framework events.

5. Suspension — Gating a Record Change on Approval

Suspension is the mechanism that holds a database change in limbo until a condition is met (typically a manager's approval). Only three framework events support suspension:

- **IdoOnItemInsert**
- **IdoOnItemUpdate**
- **IdoOnItemDelete**

When a handler for one of these events has **Suspend = True**, the system follows a two-phase process:

Phase	What happens
Suspend-Validating Mode	The system begins a transaction, performs the insert/update/delete to validate SQL constraints, runs synchronous non-suspended handlers, then rolls back the transaction. The record is locked (InWorkflow = 1) so no one else can change it. A task is queued for the commit phase.
Suspend-Committing Mode	The Event Service picks up the queued task. It runs all handlers (including those marked Suspend). If all succeed, the change is committed to the database and InWorkflow is cleared. If any handler fails, the change is abandoned and InWorkflow is cleared.

Practical use: Approval workflows. A new purchase order is created → IdoOnItemInsert fires with Suspend = True → a Prompt action sends the manager an approval request → the PO record is locked → when the manager approves, the commit phase runs and the PO is saved. If the manager rejects, a Fail action prevents

the commit.

6. Key Forms of the Application Event System

Navigate to most AES forms via: Master Explorer > Modules > System > Event System.

Form	What you do here
Access As	View-only. Shows your current ownership identifier.
Events	Name events. An event defined here but not on Event Triggers or Event Handlers is inert.
Event Triggers	Define time/condition-based triggers that fire a custom event. Cannot trigger framework events here.
Event Handlers	The primary work form. Create handlers: set Event Name, Applies To Objects, Synchronous checkbox, Suspend checkbox, Keep With / Chronology (ordering).
Event Actions	Linked from Event Handlers. Define the sequence of actions for a handler. Opened via the 'Event Actions' button on the handlers form.
Event Variable Groups	Define initial state values (variable name + starting value) to pass to a handler when it starts executing.
Event Global Constants	Named static values referenced in event expressions (GC(ConstantName)). Change the constant once and all handlers that reference it update.
Event Handler Diagram	Visual flowchart of a handler's action sequence. Drag-and-drop to add/remove actions; double-click to edit.
New Workflow Wizard	Guided wizard for simple 'Notify Me When' or 'Notify You When' notification workflows — no manual handler/action creation needed.
Workflow Event Handler Activation	50+ predefined handlers (alert on late shipment, approve credit limit change, etc.). Fill in recipients and activate.
Event Status	Track running and completed events. View handlers, parameters, voting status, output parameters.
Event Handler Status	Drill into a specific handler's run: Actions tab (which actions ran), Variables tab (current values), Voting tab (approval status).
Event Queue	View asynchronous events waiting to be processed (FIFO order).
Event Revisions / Handler Revisions	Read-only history of metadata snapshots. Running events execute against the revision in effect when they started, not the current metadata.
Suspended Updates	List of records locked in InWorkflow = 1, waiting for handler completion.
Inbox	Where system messages and approval prompts appear for the recipient. Response tab shows voting buttons; Variables tab shows payload data.

Form	What you do here
Session Access As	Enable/disable handlers by Access As identifier for troubleshooting. Useful for isolating whether a custom handler is causing a bug.

7. Creating a Custom Event and Handler — Step by Step

Step	Action
1. Name the event	Events form — add a row with a unique event name (optional; you can also name it on the handler form directly).
2. Define the handler	Event Handlers form → New. Set: Event Name, Handler Description, Applies to Objects (filter to a specific IDO, e.g. SLCustomers), Synchronous checkbox (clear for async notifications), Suspend checkbox (check if you need approval gating).
3. Add event actions	Click 'Event Actions' on the handler. For each action: set Action Sequence number (10, 20, 30...) and Action Type.
4. Set action parameters	Click 'Edit Parameters' to open the type-specific parameter form. Build expressions using the Expression Editor. Click 'Check Syntax' before saving.
5. Create global constants	Event Global Constants form — define reusable values (e.g., CreditMgr = 'jsmith'). Reference with GC(ConstantName) in expressions.
6. Discard metadata cache	Ctrl+U (with 'Unload IDO Metadata With Forms' on) — ensures the running system sees your new handler.
7. Test	Trigger the event (perform the action on the relevant form). Check Event Status form or Inbox to verify.

8. Event Action Parameters — Key Functions

Action parameters are written in an expression language using FUNCTION(value) syntax. Key function categories:

Value retrieval functions

Function	Returns
V(varName)	Value of an event variable.
GC(constantName)	Value of an event global constant.
E(paramName)	Value of an event parameter.
P(propertyName) or PROPERTY(propertyName)	Value of a framework event property (the data being inserted/updated/deleted).
FP(propertyName) or FILTERPROPERTY(propertyName)	Value as a quoted string (for use in filter expressions).
SV(varName)	Value of a WinStudio session variable.

Function	Returns
ORIGINATOR()	User ID of the person whose action triggered the event.
CURDATETIME()	Current date and time (site-adjusted).
PROPERTYMODIFIED(propertyName)	TRUE if the property was changed in this update (IdoOnItemUpdate only).

Text construction functions

Function	Use
SUBSTITUTE('text with {0} and {1}', expr0, expr1, ...)	Builds a string by replacing {0}, {1} etc. with evaluated expressions. Most common way to build notification message bodies.
CLIENTSUBSTITUTE(...)	Same as SUBSTITUTE but the result is stored with localization markers so each Inbox user sees the string in their configured language.

Voting / prompt result functions

Function	Returns
VOTINGRESULT(actionSequenceNum)	Winning choice value from the specified Prompt action.
VOTINGTIE(actionSequenceNum)	TRUE if the vote resulted in a tie.
VOTINGDISPARITY(actionSequenceNum)	TRUE if any two recipients selected different choices.
RESPONDERS(actionNum [, choice])	Count of recipients who responded (optionally for a specific choice).
RECIPIENTS(actionNum)	Total number of recipients who received the message.

9. Requesting Approvals — Basic Pattern

The most common AES workflow: send an approval request and suspend the record until the response is received.

Required action sequence for a basic approval

Action #	Type	Purpose
10	Prompt	Sends the approval request to the manager. Key parameters: TO(recipient), SUBJECT(...), BODY(...), QUESTION('Do you approve?'), CHOICES('1,sYes,0,sNo'), VOTINGRULE(Plurality).
20	Fail (conditional)	If the manager voted No: CONDITION(VOTINGRESULT(10) = '0'), RESULT('Rejected by manager.'). This prevents the record from committing.

Handler-level setting: Check the **Suspend** checkbox on the handler so the record is held in InWorkflow=1 until the handler completes.

Responding to a prompt (manager's perspective): Open Inbox → open message → click Response tab → click Approve or Reject button.

10. Voting Rules for Multi-Person Approvals

When a Prompt is sent to multiple recipients, set the VOTINGRULE() parameter to define how the system tallies responses.

Rule	Behavior
Majority	First choice to receive > 50% of votes wins. Recommend using VOTINGTIE() for tie handling.
Plurality	Choice with the most votes wins, even if < 50%.
ConditionalPlurality	Plurality, but only if the winner reaches a specified MINIMUM() percentage.
MinimumCount	First choice to reach a specified MINIMUM() vote count wins immediately.
MinimumPercentage	First choice to reach a specified MINIMUM() percentage of all recipients wins immediately.
EarliestResponse	First response received wins, regardless of choice. Voting closes immediately.
PreferredChoice	If anyone votes for the designated PREFCHOICE(), that choice wins immediately. Otherwise falls back to Plurality.

Quorum: The system automatically calculates the minimum number of votes needed to determine a winner. Once the quorum is reached, unvoted messages are expired and voting closes. Override with QUORUM(n) if needed.

11. Tracking Event System Status

Form	Use it when...
Event Status	An event fired but you're not sure what happened. Look for failures, check parameters passed, see all handlers that ran.
Event Handler Status	A specific handler didn't do what you expected. Actions tab shows which actions started. Variables tab shows variable values at failure.
Event Queue	Checking whether asynchronous events are piling up or stalled.
Event Revisions / Handler Revisions	Understanding why an old version of a handler ran (handlers run against the revision in effect when they started).
Suspended Updates	Finding a record that is stuck in InWorkflow=1 (approval pending with no response received).

Troubleshooting the event service not running:

- Verify the Infor Framework Event Service is Running (Windows Services on the utility server).

- Verify your configuration is listed on the Event Service tab of the Service Configuration Manager.
 - Check the Infor Framework Log Monitor for error messages.
-

12. The Infor Framework Event Service

Asynchronous events and suspended events are processed by the **Infor Framework Event Service** — an independent Windows service on the utility server.

- Processes queued events in FIFO (first-in, first-out) order.
- After each queued item, checks for adjourned handlers whose retest/timeout time has arrived.
- Must be configured with your CloudSuite configuration (site name) in the Service Configuration Manager.
- If this service is not running, all asynchronous events and all suspended-commit processing will stall.
- The service can run on multiple servers for load-balancing; uses database locking to prevent double-processing.

Setting up the service

- On the utility server: Start > Infor CSI > Service Configuration Manager (run as administrator).
 - Click the Event Service tab > Add > select your configuration > Save.
 - If you added a new configuration, restart the Infor Framework Event Service for it to take effect.
-

13. Setting Up External Email

The AES can send messages to recipients' external email (Outlook, etc.) in addition to their WinStudio Inbox.

Step	Where
1. Get SMTP settings	From the email administrator.
2. Configure SMTP	Intranets form (System administrator) → Reporting tab → SMTP section.
3. Add user email address	Users form → Email Address tab → Primary email, set as Primary Email.
4. Enable send for user	Users form → Email Address tab: check 'Send External Notifications' and/or 'Send External Prompts'.

Action type	Sends to Inbox?	Sends to external email?
Notify	Yes	Yes (if user has external notifications enabled)
Prompt	Yes	Yes (if user has external prompts enabled) — but free-form responses not supported via email
Send Email	No	Yes — only sends to external email

External prompt responses: Recipients click a hyperlink in the email that opens a browser page and records their vote. If they respond via email first, the Inbox message is marked expired so they cannot vote twice.

Quick Reference: Common Expression Patterns

Pattern	Expression
Notify a global constant (role)	TO(GC(CreditMgr))
Notify the person who made the change	TO(ORIGINATOR())
Message body with substitution	BODY(SUBSTITUTE('Credit limit changed to \${0} for {1}', FP('CreditLimit'), P('Name')))
Check if a property was modified	CONDITION(PROPERTYMODIFIED('CreditLimit'))
Check if a property was NOT modified	CONDITION(NOT PROPERTYMODIFIED('CreditLimit'))
Filter to a specific form	FILTERFORM('Customers') FILTER(SUBSTITUTE('CustNum={0}', FP('CustNum')))
Yes/No voting choices	CHOICES('1,sYes,0,sNo') QUESTION('Do you approve?')
Fail if manager voted No	CONDITION(VOTINGRESULT(10) = '0') RESULT('Rejected.')
Expand a global constant textually (pre-parser)	TGC(GlobalConstantName) — expands value before parsing
Load old value before update (sync handler)	Load IDO Row action with IDO(SLItems) PROPERTIES('LotSize') FILTER(SUBSTITUTE('Item={0}',FP('Item'))) SET(RV(OldValue)='LotSize')